

FORMAL LANGUAGES AND COMPILERS

prof.s Luca Breveglieri and Angelo Morzenti

Exam of Tue 15 JANUARY 2019 - Part Theory

LAST + FIRST NAME:

(capital letters please)

MATRICOLA:

(or PERSON CODE)

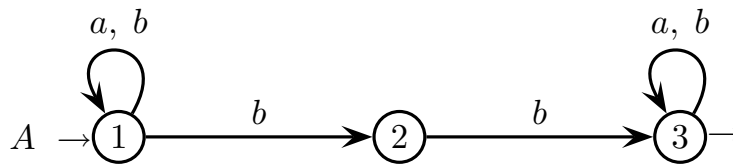
SIGNATURE:

INSTRUCTIONS - READ CAREFULLY:

- The exam is in written form and consists of two parts:
 1. Theory (80%): Syntax and Semantics of Languages
 - regular expressions and finite automata
 - free grammars and pushdown automata
 - syntax analysis and parsing methodologies
 - language translation and semantic analysis
 2. Lab (20%): Compiler Design by Flex and Bison
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- To pass part theory, the candidate must answer the mandatory (not optional) questions; notice that the full grade is achieved by answering the optional questions.
- The exam is open book: textbooks and personal notes are permitted.
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.
- Time: part lab 60m - part theory 2h.15m

1 Regular Expressions and Finite Automata 20%

1. Consider the nondeterministic finite-state automaton A below, over a two-letter alphabet $\Sigma = \{ a, b \}$:



Answer the following questions:

- (a) Find a deterministic version A' of automaton A , by using the Berry-Sethi (BS) method.
 - (b) By using a systematic method, determine whether the deterministic automaton A' is minimal, and if it is not so, find the equivalent minimal automaton.
 - (c) By using a systematic method, find an automaton $\overline{A}$ that accepts language $\overline{L}$, i.e., the complement of language $L(A)$.
 - (d) (optional) Find a regular expression that generates the complement language $\overline{L}$, by applying the Brzowski-McCluskey (BMC) method to automaton $\overline{A}$.
-

2 Free Grammars and Pushdown Automata 20%

1. Consider the context-free language L below, over a three-letter alphabet $\Sigma = \{ a, b, c \}$:

$$L = \{ a^n (b c^{k_i})^n \mid n \geq 0 \text{ and } \forall i \text{ with } 1 \leq i \leq n \text{ it holds } k_i \geq 1 \}$$

Sample valid strings are:

$$\varepsilon \qquad a b c \qquad a b c c \qquad a a b c b c \qquad a a b c c b c$$

Notice that the number of consecutive letters c may differ in each substring $b c^{k_i}$.

Answer the following questions:

- (a) Write a *BNF* grammar G , not ambiguous, that generates language L . Test grammar G by drawing the syntax tree of the last sample string listed above:

$$a a b c c b c$$

- (b) Formally prove or at least convincingly argue that grammar G is not ambiguous.
(c) (optional) Consider the context-free language L' below:

$$L' = L L^R$$

where language L^R is the mirror image of language L . Write a set representation of language L^R , by changing that of L . Then write a *BNF* grammar G' , not ambiguous by construction, that generates language L' , and explain why the construction is not ambiguous.

2. Consider a script language that defines simple regular expressions, like for instance:

$$e_1 = (a \mid b)^+ \cdot (c \cdot [d])^*$$

according to the following rules and constraints:

- an expression may include the following n -ary operators (with $n \geq 2$): alternative “ $\mid$ ” and concatenation “ $\cdot$ ”, like $a \mid b \mid \dots$ and $a \cdot b \cdot \dots$
- an expression may include the following unary operators: Kleene star iterator “ $*$ ”, cross iterator “ $+$ ” and optionality “[]”
- the immediate nesting of any operator into itself, like in $(a \mid (b \mid c))$, is forbidden
- the immediate nesting of any two unary operators, like in $(b^+)^*$, is forbidden
- an expression name, like the name e_1 above, is an instance of the grammar terminal symbol `id`
- the alphabetical symbols of an expression, like the letters a , b , c and d in the expression e_1 above, are instances of the grammar terminal symbol `sym`
- a language phrase consists of a non-empty list of expression definitions separated by semicolons “ $;$ ”, and the whole list is terminated by a dot “ $\cdot$ ”

Here is a short sample script, which defines the sample expression e_1 above:

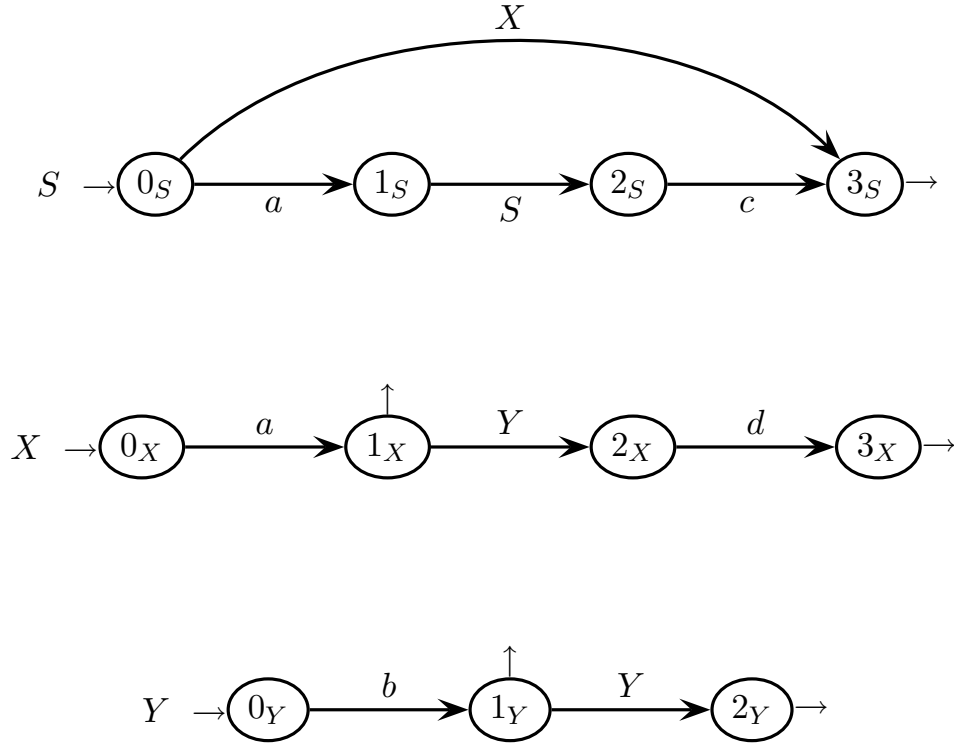
```
regexp e1 is
  conc
    cross
      alt
        a, b
      endalt
    endcross,
  star
    conc
      c,
    opt
      d
    endopt
  endconc
endstar
endconc
endregexp.
```

You are invited to infer from this sample script any information that is not explicitly provided by the rules and constraints listed above.

Write a grammar, possibly of type *EBNF* and not ambiguous, that generates the script language described above.

3 Syntax Analysis and Parsing Methodologies 20%

1. Consider the following grammar G , represented as a machine net, over a four-letter terminal alphabet $\Sigma = \{ a, b, c, d \}$ and a three-symbol nonterminal alphabet $V = \{ S, X, Y \}$ (axiom S):



Answer the following questions (use the figures / tables / spaces on the next pages):

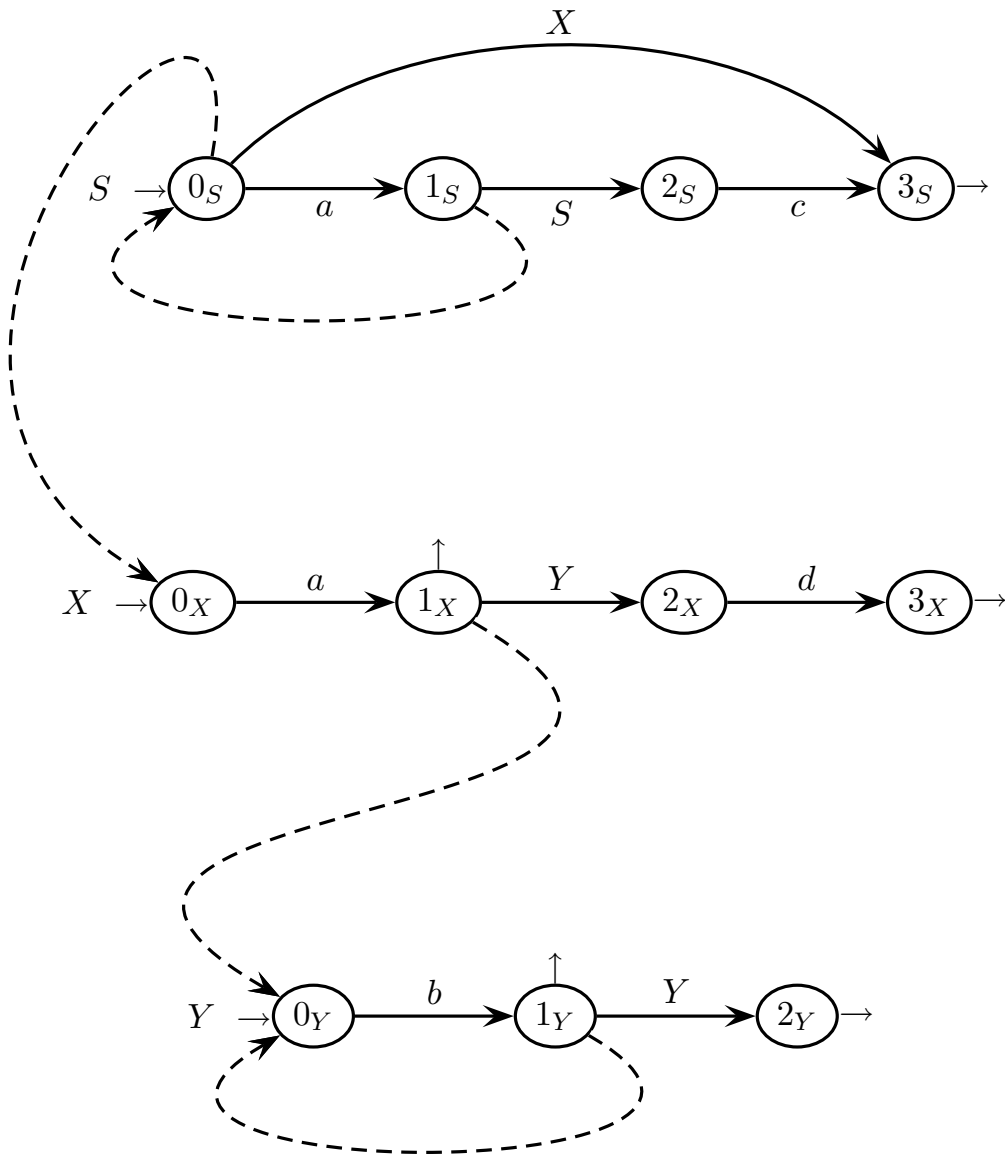
- (a) Draw the complete pilot of grammar G (with all the m-states), determine whether grammar G is $ELR(1)$ and explain why yes or no.
- (b) Write all the guide sets of the machine net of grammar G (on the call arcs and exit arrows), show that grammar G is *not* $ELL(1)$ and list all the guide set conflicts that are present.
- (c) Determine whether grammar G is $ELL(2)$ and explain why yes or no.
- (d) (optional) Give a representation of language $L(G)$ as a set of strings, like:

$$L(G) = \{ \text{string model} \mid \text{characteristic predicate} \}$$

In a convincing way, argue whether language $L(G)$ is regular or not.

question (a) – please draw here the pilot of grammar G

question (b) – please write here all the guide sets on the call arcs and exit arrows



space for questions (c) and (d)

4 Language Translation and Semantic Analysis 20%

1. Consider the following source grammar G_s , over a four-letter terminal alphabet $\Sigma = \{ a, c, d, e \}$ and a two-symbol nonterminal alphabet $V = \{ S, P \}$ (axiom S):

$$G_s \left\{ \begin{array}{l} S \rightarrow P S \\ S \rightarrow P \\ P \rightarrow d P a \\ P \rightarrow e P a \\ P \rightarrow c \end{array} \right.$$

which generates the source language L_s below:

$$L_s = \{ ((d \mid e)^{k_i} c a^{k_i})^n \mid n \geq 1 \text{ and } \forall i \text{ with } 1 \leq i \leq n \text{ it holds } k_i \geq 0 \}$$

Answer the following questions (and do not change the source grammar G_s):

- (a) Write a translation scheme that defines a translation function τ as follows (the symbols w_i indicate the substrings of letters d and e):

$$\tau(w_1 c a^{|w_1|} \dots w_n c a^{|w_n|}) = a^{|w_1|} f w_1^R \dots a^{|w_n|} f w_n^R$$

For instance (here $n = 2$ and $k_1 = 3, k_2 = 2$):

$$\tau(\underbrace{e d d}_{w_1} c a a a \underbrace{d e}_{w_2} c a a) = a a a f \underbrace{d d e}_{w_1^R} a a f \underbrace{e d}_{w_2^R}$$

- (b) Determine whether translation τ can be computed deterministically by an $ELL(1)$ or $ELR(1)$ parser enriched with suitable print actions, and motivate your answer. In the positive case, write the pseudo-code of the integrated syntax analysis and translation procedure for nonterminal P .
- (c) Write a translation scheme that defines the inverse translation function τ^{-1} .
- (d) (optional) Determine whether the translation function τ^{-1} can be computed deterministically by an $ELL(1)$ or $ELR(1)$ parser enriched with suitable print actions, and motivate your answer.

- $$\left\{ \begin{array}{l} S \rightarrow x : (cL:) (\text{---} S \text{---}) \\ L \rightarrow L, E \\ L \rightarrow E \\ E \rightarrow (L) \\ E \rightarrow y \end{array} \right.$$

Answer the following questions (use the tables / spaces on the next pages):

- | <i>data structure</i> | <i>exists</i> |
|-----------------------|-------------------------|
| $a : (b, a, c)$ | yes |
| $c : (b, (c, a), c)$ | yes (sample tree above) |
| $a : (b, b, c)$ | no |

(b) Write an attribute grammar that computes the depth of the first, i.e., leftmost, occurrence of object x in the data structure, if any. Notice that the depth is the number of enclosing brackets, not the distance from the tree root. Sample cases:

<i>data structure</i>	<i>object depth (if any)</i>
$a : (b, a, c)$	1
$c : (b, (c, a), c)$	2 (sample tree above)
$a : (b, b, c)$	0 (inexistent object)

(c) (optional) For the one-sweep attribute grammar of question (a), write the procedure L of the semantic analyzer. Specify the formal and actual parameters of L , as well as those of the procedure E invoked by L . With what actual parameters should procedure L be invoked in the axiomatic procedure S ?

attribute specifications – questions (a) and (b)

<i>name</i>	<i>type</i>	<i>domain</i>	<i>symbol</i>	<i>meaning</i>
<i>e</i>	left	boolean	S, L, E	<i>true</i> if object x occurs somewhere in the data (sub)structure rooted at the current node; else <i>false</i>
<i>o</i>	right	object	L, E	the object x to look for in the data (sub)structure
<i>d</i>	right	integer ≥ 1	L, E	the <i>depth</i> (≥ 1) of an element (sublist or object) in the data (sub)structure
<i>f</i>	left	integer ≥ 0	S, L, E	the <i>depth</i> (≥ 1) of the first (leftmost) instance of object x that is found in the data (sub)structure, if any; else 0

syntax *semantics* – question (a)

1: $S_0 \rightarrow x : (L_1)$

2: $L_0 \rightarrow L_1 , E_2$

3: $L_0 \rightarrow E_1$

4: $E_0 \rightarrow (L_1)$

5: $E_0 \rightarrow y$

syntax *semantics* – question (b)

1: $S_0 \rightarrow x : (L_1)$

2: $L_0 \rightarrow L_1 , E_2$

3: $L_0 \rightarrow E_1$

4: $E_0 \rightarrow (L_1)$

5: $E_0 \rightarrow y$

space for question (c)

free space for continuation or notes on all exercises

free space for continuation or notes on all exercises